

Algorithms, AI & Data Science II

Prof. Dr. Ingo Scholtes

Chair of Machine Learning for Complex Networks
Center for Artificial Intelligence and Data Science (CAIDAS)
Julius-Maximilians-Universität Würzburg
ingo.scholtes@uni-wuerzburg.de

Motivation

- ▶ we introduced Markov models of stochastic processes
- ▶ we explored under which conditions Markov chains reach a **unique stationary state**
- ▶ stationary states are basis for important **algorithmic applications of Markov chains**
- ▶ we can use stationary state of **random surfer in Web graph** to rank Web pages based on **relevance or importance**
- ▶ we can use Markov chains for the **heuristic exploration of large state spaces**



DALL-E image generated for prompt "searching a needle in a haystack"

image credit: www.bing.com

Notes:

- **Lecture 16: Markov Chain Monte Carlo Algorithms** 02.07.2025
- **Educational objective:** We explore Markov Chain Monte Carlo algorithms. We show how Markov chains can be used to sample from arbitrary distributions, and how we can use physics-inspired simulated annealing to address NP-hard optimization problems.
 - Markov Chain Monte Carlo
 - Metropolis Sampling
 - Optimization by Simulated Annealing
- **Handout version:** 2025/06/30 13:37:37

Notes:

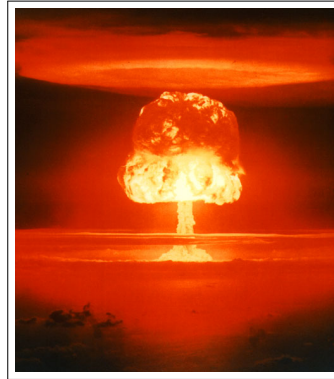
Dealing with large state spaces

- ▶ we often face algorithmic problems in **huge state spaces** that are too large to enumerate
 - ▶ NP-hard optimization problems
 - ▶ sampling from state spaces with complex distributions

application: computational physics

- ▶ statistical physics studies **macroscopic properties of materials** comprised of large number of interacting particles
- ▶ probabilities of **specific particle arrangements** depend on temperature, energy, etc.
- ▶ **huge space** of possible arrangements of a very large number of particles (e.g. all possible positions/velocities of $\approx 10^{23}$ atoms)

- ▶ how can we **efficiently sample from huge state spaces**?

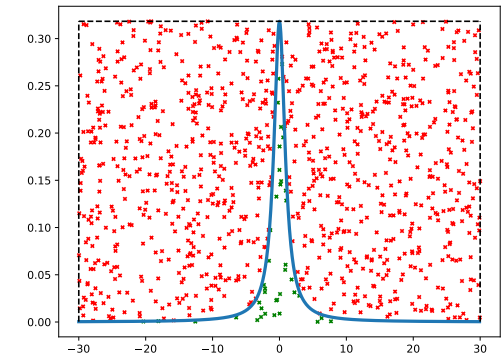


Castle Romeo test of a hydrogen bomb

image credit: United States Department of Energy, public domain

Rejection sampling?

- ▶ in L09, we introduced **rejection sampling algorithm**
- ▶ can be used to sample from **arbitrary (continuous) distributions**
- ▶ is this **efficient** for any distribution?
- ▶ consider complex distribution where **probabilities are concentrated in small portions of state space**
- ▶ we **reject huge number of samples**
- ▶ can we **use Markov chains** to derive a more efficient sampling technique?



accepted (green) and rejected (red) samples generated by **rejection sampling algorithm**

Notes:

Notes:

Markov Chain Monte Carlo

- ▶ for given transition matrix $\mathbf{T}$ of Markov chain, **we can calculate stationary distribution π of states**
- ▶ **idea:** define Markov model such that for $t \rightarrow \infty$ state probabilities converge to **desired unique target distribution π**

general approach

- ▶ start from random initial state
 - ▶ simulate random walk through state space
 - ▶ consider state that we arrive at after sufficiently large number of steps
-
- ▶ basis for **Markov Chain Monte Carlo algorithms** widely used in data science, statistics, computational physics, or bioinformatics



Casino of Monte Carlo

image credit: Wikimedia Commons, Fruitpunchline, CC-BY-SA

Notes:

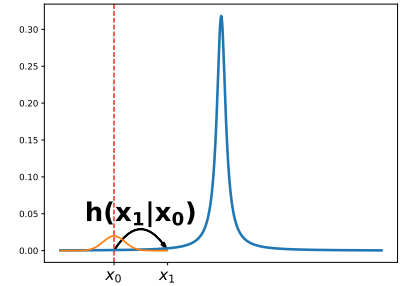
Metropolis sampling

- ▶ consider that we know function $f(x)$ and want to sample from **target distribution π with $\pi_x \propto f(x)$** for states x
- ▶ we pick a random initial state x_0
- ▶ assuming that we are in state x_t , consider iterative algorithm that samples new state x_{t+1} from **symmetric proposal distribution $g(x_{t+1}|x_t) = g(x_t|x_{t+1})$**
- ▶ we accept new state x_{t+1} with probability

$$h(x_{t+1}|x_t) := \min\left(1, \frac{f(x_{t+1})}{f(x_t)}\right)$$

or stay in current state x_t

- ▶ for $t \rightarrow \infty$ **Metropolis algorithm samples states x with probabilities π_x**



Metropolis algorithm → N Metropolis et al., 1953

```
procedure METROPOLIS_SAMPLING( $f, g, N$ )  
   $x \leftarrow \text{initial\_state}()$   
  for  $i$  in range( $N$ ) do  
     $x_{\text{new}} \leftarrow \text{sample}(g, x)$   
     $p \leftarrow \text{uniform}(0, 1)$   
    if  $p \leq \min(1, f(x_{\text{new}})/f(x))$  then  
       $x \leftarrow x_{\text{new}}$   
    end if  
  end for  
  return  $x$   
end procedure
```

Notes:

Metropolis sampling as Markov chain

- ▶ for a discrete state space, let us consider the **Markov chain defined by Metropolis algorithm**
- ▶ since the generation and acceptance of proposal are independent, we obtain the following **entries of the transition matrix T** of the Markov chain

$$T_{x_t, x_{t+1}} = g(x_{t+1}|x_t) \cdot \min\left(1, \frac{f(x_{t+1})}{f(x_t)}\right) \quad T_{x_{t+1}, x_t} = g(x_t|x_{t+1}) \min\left(1, \frac{f(x_t)}{f(x_{t+1})}\right)$$

- ▶ since the **proposal distribution is symmetric**, i.e. $g(x_{t+1}|x_t) = g(x_t|x_{t+1}) =: \epsilon$, we have

$$T_{x_t, x_{t+1}} = \epsilon \cdot \min\left(1, \frac{f(x_{t+1})}{f(x_t)}\right) \quad T_{x_{t+1}, x_t} = \epsilon \cdot \min\left(1, \frac{f(x_t)}{f(x_{t+1})}\right)$$

- ▶ for states x_t and x_{t+1} with $f(x_{t+1}) > f(x_t)$ we then obtain

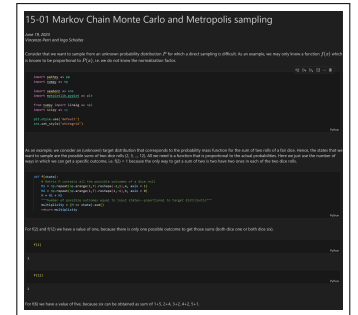
$$T_{x_t, x_{t+1}} = \epsilon \quad \text{and} \quad T_{x_{t+1}, x_t} = \epsilon \cdot \frac{f(x_t)}{f(x_{t+1})} < \epsilon$$

i.e. ϵ is probability to move to state x_{t+1} with “larger” value $f(x_{t+1})$

Notes:

Practice Session

- ▶ we use the Metropolis sampling algorithm to sample from the probability mass function of a random variable X assuming the sum of two rolls of a fair dice
- ▶ we use a random walk in an arbitrary connected graph to generate a proposal distribution
- ▶ as function $f(x)$ we use the number of possible combinations in which can obtain a sum of x
- ▶ we calculate the stationary state of the resulting Markov chain and show that it converges towards to desired target distribution



practice session

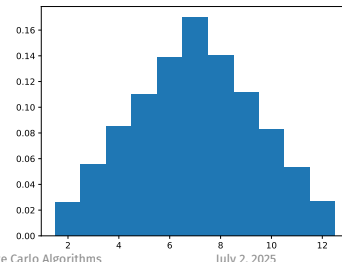
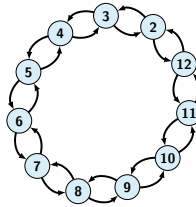
see notebook 16-01 in [gitlab repository](https://gitlab.informatik.uni-wuerzburg.de/ml4nets_notebooks/2025_sose_akids2) at

→ https://gitlab.informatik.uni-wuerzburg.de/ml4nets_notebooks/2025_sose_akids2

Notes:

Analysis of Metropolis sampling

- ▶ for our simple example, we experimentally confirmed that the Metropolis sampling algorithm converges to “correct” stationary state π with $\pi_i \propto f(i)$
- ▶ but **why** does this actually work?
- ▶ we know that a unique stationary state exists if Markov chain is **irreducible and aperiodic** → L15
- ▶ need to show that target distribution π is **stationary state of Markov chain** defined by Metropolis algorithm
- ▶ proof idea: symmetry of proposal distribution allows us to show a strong property that yields **sufficient condition for stationary state**



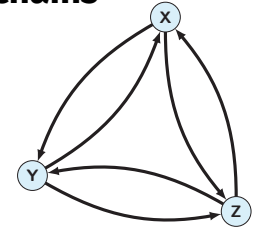
Detailed balance and reversible Markov chains

- ▶ we say that Markov chain with transition matrix $\mathbf{T}$ is **reversible** (satisfies the **detailed balance condition**) iff a stationary state π exists such that

$$\forall i, j : \pi_i T_{ij} = \pi_j T_{ji}$$

- ▶ for a reversible Markov chain, we can **reverse time** and obtain the same stochastic process
- ▶ detailed balance holds for $\mathbf{T}$ and $\pi \implies \pi$ stationary state for $\mathbf{T}$
- ▶ π stationary state for $\mathbf{T} \not\implies$ detailed balance holds for $\mathbf{T}$ and π
- ▶ i.e. detailed balance condition is **sufficient but not necessary condition** for π being stationary state of $\mathbf{T}$

→ exercise sheet



$$\mathbf{T} = \begin{matrix} & \begin{matrix} x & y & z \end{matrix} \\ \begin{matrix} x \\ y \\ z \end{matrix} & \begin{bmatrix} 0 & \frac{1}{2} & \frac{1}{2} \\ \frac{1}{2} & 0 & \frac{1}{2} \\ \frac{1}{2} & \frac{1}{2} & 0 \end{bmatrix} \end{matrix}$$

example

for $\pi = (\frac{1}{3}, \frac{1}{3}, \frac{1}{3})$ and any pair i, j we have

$$\pi_i \cdot T_{ij} = \frac{1}{3} \cdot \frac{1}{2} = \pi_j \cdot T_{ji}$$

Notes:

Notes:

Group Exercise 16-01

1/2

- Show that the Markov chain defined by the Metropolis algorithm satisfies the detailed balance condition for a stationary state π with $\pi_i \propto f(i)$.

- Consider two states x and y . Since the proposal and acceptance are independent, we can write the transition probability as product of the proposal probability $g(y|x)$ and the probability of acceptance of the proposal $h(y|x) = \min(1, \frac{f(y)}{f(x)})$, i.e.

$$T_{x,y} = g(y|x) \cdot h(y|x)$$

- Similarly, the probability of moving from y to x can be written as:

$$T_{y,x} = g(x|y) \cdot h(x|y)$$

- For stationary state π with $\pi_i = \alpha \cdot f(i)$ for any state i , the detailed balance condition corresponds to:

$$\alpha f(x) \cdot T_{x,y} = \alpha f(y) \cdot T_{y,x}$$

$$f(x) \cdot g(y|x) \cdot h(y|x) = f(y) \cdot g(x|y) \cdot h(x|y)$$

Group Exercise 16-01

2/2

- The Metropolis algorithm assumes a symmetric proposal distribution, i.e., $g(y|x) = g(x|y)$ which allows us to simplify the equation above to:

$$f(x) \cdot \cancel{g(y|x)} \min\left(1, \frac{f(y)}{f(x)}\right) = f(y) \cdot \cancel{g(y|x)} \cdot \min\left(1, \frac{f(x)}{f(y)}\right)$$

- We can rewrite this as

$$\frac{f(x)}{f(y)} = \frac{\min(1, \frac{f(x)}{f(y)})}{\min(1, \frac{f(y)}{f(x)})}$$

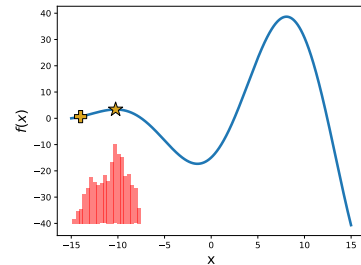
- Finally, it is easy to verify that this holds for $f(x) > f(y)$, for $f(y) > f(x)$ and $f(x) = f(y)$ □

Notes:

Notes:

From sampling to heuristic optimization

- ▶ for given function $f(x)$, we can define **Markov chain** whose stationary state is proportional to $f(x)$
- ▶ can we define a Markov chain whose stationary state corresponds to **maximum value of $f(x)$** ?
- ▶ basis for **metaheuristic optimization algorithms**
- ▶ for given $f(x)$ consider distribution of states sampled by Metropolis algorithm for **finite number of steps**
- ▶ since we are more likely to move to states with larger $f(x)$ we are **likely to be “stuck” in local maxima**
- ▶ how can we **explore the state space** and still converge to an optimum?



distribution of sampled states of **Metropolis Markov Chain Monte Carlo** for $N = 10000$ steps
(initial and maximum state marked)

Markov chains in statistical physics

- ▶ statistical physics studies how **macroscopic properties of aggregate matter** can be understood based on statistics of underlying particle arrangements

examples

- ▶ properties of water depending on temperature and pressure
- ▶ melting point of iron
- ▶ **state of the system** is described by specific configuration of particles (i.e. atoms or molecules)
- ▶ random fluctuations depend on **energy of states** and **temperature of the system**
- ▶ particles are memoryless, i.e. **transitions between states** can be modelled using **Markov chain**



Eruption of Castel Geyser in Yellowstone National Park

image credit: Wikimedia Commons, Brocken Inaglory, CC-BY-SA

Notes:

Notes:

A statistical physics perspective on optimization

- **Boltzmann distribution** describes how probabilities of particle arrangements x depend on their **energy** $E(x)$ and **temperature** T

$$P(x) \propto e^{-\frac{E(x)}{T}}$$

- transition probability from x to y given by **Boltzmann ratio**

$$\frac{P(y)}{P(x)} = e^{-\frac{E(y)-E(x)}{T}}$$

observations

- for $T \rightarrow \infty$ we have $e^{-\frac{E(y)-E(x)}{T}} \rightarrow e^0 = 1$
- for $T \rightarrow 0$ we have $e^{-\frac{E(y)-E(x)}{T}} \rightarrow \infty$ for $E(x) > E(y)$ and 0 otherwise
- at low temperature systems tend to move to states with **lower energy**



Ludwig Boltzmann, 1844 – 1906

image credit: public domain



blacksmith annealing metal

image credit: Jeff Kubina from Columbia, Maryland, CC-BY-SA

Notes:

Optimization by simulated annealing

- transitions based on Boltzmann ratio assign high probabilities to states with **lowest energy**
- to find **maximum of function** f we can simulate “annealing”, i.e. we accept state x_{t+1} with probability

$$h(x_{t+1}|x_t) = \min\left(1, e^{-\frac{f(x_t)-f(x_{t+1})}{T}}\right)$$

- we swap $f(x_t)$ and $f(x_{t+1})$ as we are interested in **maximum of f** (rather than minimum of energy)
- to explore state space and escape local maxima, we start algorithm with **high temperature**
- **cooling schedule** $T \rightarrow 0$ allows Markov chain to “settle” in states with maximum $f(x)$
- we call this algorithm **simulated annealing**



Scott Kirkpatrick

image credit: <https://www.cs.huji.ac.il/~w-kirk/>

simulated annealing algorithm → S Kirkpatrick et al., 1983

```
procedure SIMULATED_ANNEALING( $f, g, N, T$ )  
   $x \leftarrow \text{initial\_state}()$   
  for  $i$  in range( $N$ ) do  
     $x_{\text{new}} \leftarrow \text{sample}(g, x)$   
     $p \leftarrow \text{uniform}(0, 1)$   
    if  $p \leq \min\left(1, e^{-\frac{f(x_t)-f(x_{t+1})}{T[i]}}\right)$  then  
       $x \leftarrow x_{\text{new}}$   
    end if  
  end for  
end procedure
```

Notes:

Practice Session

- ▶ we implement the simulated annealing algorithm in python
- ▶ we use it to address a simple optimization problem
- ▶ we explore the role of the temperature parameter in simulated annealing

```
15-02 Optimization by Simulated Annealing
Date: 15.02.2025
Author: Lisi Qarkaxhija

This notebook shows how to implement the simulated annealing algorithm, a Markov Chain Monte Carlo approach for function optimization, and demonstrate it on a very simple example for a 2D optimization problem.

We will optimize a simple function f(x,y) = 10 + x^2 + y^2, which is a simple paraboloid. The goal is to find the minimum of this function, which is at (0,0).

We will use the simulated annealing algorithm, which is a stochastic optimization algorithm. It starts from a random initial state and iteratively moves to a new state, accepting or rejecting the move based on a probability that depends on the change in the objective function and the current temperature. The temperature decreases over time, allowing the algorithm to escape local minima and converge to the global minimum.

We will also visualize the distribution of sampled states, which shows the path of the algorithm and the regions where it spends most of its time. The distribution should be concentrated around the global minimum at (0,0).

We will use the following code to implement the algorithm and visualize the results.
```

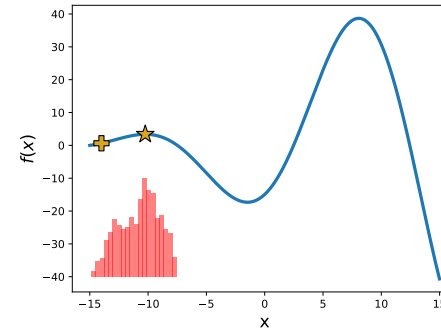
practice session

see notebook 16-02 in gitlab repository at

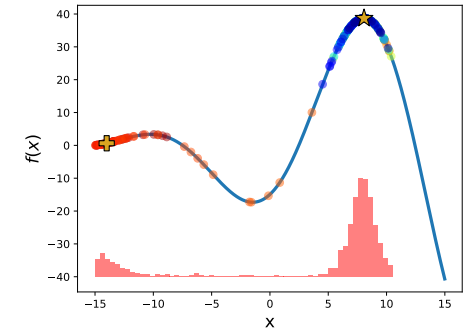
→ https://gitlab.informatik.uni-wuerzburg.de/ml4nets_notebooks/2025_sose_akids2

Notes:

Example



distribution of sampled states of **Metropolis Markov Chain Monte Carlo** for $N = 10000$ steps
(initial and maximum state marked)

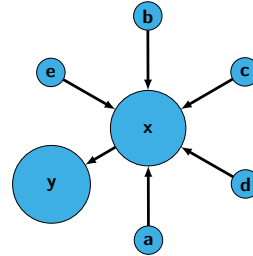


distribution of sampled states of **simulated annealing algorithm** for $N = 10000$ steps
(color represents temperature T)

Notes:

In summary ...

- ▶ Markov chains are theoretical basis for **algorithms for heuristic state space exploration**
- ▶ **Markov Chain Monte Carlo** is important approach in data science, statistics, computational physics, and bioinformatics
- ▶ we can use Metropolis algorithm to **sample from arbitrary distributions** (e.g. for Bayesian inference)
- ▶ Markov chain Monte Carlo is basis for **simulated annealing**
- ▶ important class of **heuristic optimization algorithms** inspired by thermodynamics
- ▶ highlights relations between **statistical physics and combinatorial optimization**



PageRank score of nodes for $\alpha = 0.85$



blacksmith forging metal

image credit: Jeff Kubina from Columbia, Maryland, CC-BY-SA

Self-study questions

1. What is the role of the function f in Metropolis sampling?
2. What conditions have to be fulfilled for the proposal distribution $g(y|x)$ in the Metropolis algorithm?
3. Investigate the Metropolis-Hastings algorithm, a generalization of the Metropolis algorithm. How is it different from the algorithm introduced in this lecture?
4. Investigate how Metropolis sampling can be used to approximate posterior distributions in Bayesian inference.
5. What is the detailed balance condition for a Markov chain and how does it relate to the existence of a stationary state?
6. Give an example for a Markov chain with a unique stationary state that does not satisfy the detailed balance condition.
7. Show that a Markov chain with a symmetric transition matrix fulfills the detailed balance condition.
8. Give an example for a graph, where the transition matrix of a random walk is symmetric.
9. What is the Boltzmann factor and how can it be applied in heuristic optimization algorithms?
10. Explain the role of the temperature parameter in the simulated annealing algorithm.

Notes:

Notes:

References

reading list

- N Metropolis, AE Rosenbluth, MN Rosenbluth, AH Teller, E Teller: **Equation of state calculations by fast computing machines**, J Chem Phys, 1953
- WK Hastings: **Monte Carlo Sampling Methods Using Markov Chains and Their Applications**, Biometrika, Vol. 57, 1970
- S Kirkpatrick, CD Gelatt Jr, MP Vecchi: **Optimization by Simulated Annealing**, Science, 1983
- O Häggström: **Finite Markov Chains and Algorithmic Applications**, 2002
- MN Rosenbluth: **Genesis of the Monte Carlo Algorithm for Statistical Mechanics**, AIP Conference Proceedings, 2003
- RW Shonkwiler, F Mendivil: **Explorations in Monte Carlo Methods**, 2009

13 May 1983, Volume 220, Number 4598

SCIENCE

Optimization by Simulated Annealing

S. Kirkpatrick, C. D. Gelatt, Jr., M. P. Vecchi

In this article we briefly review the central constructs in combinatorial optimization in statistical mechanics and their relevance to the study of the many-body problem. We show how the Metropolis algorithm for approximate sampling of the behavior of a many-body system at a finite temperature provides a natural tool for finding the minimum of a function of a large number of variables. We have applied this point of view to a number of problems arising in statistical mechanics. Applications to design of computers, applications to scheduling, and applications to the design of electronic systems are discussed in this article. In each context, we illustrate the problem and discuss the improvements available from optimization.

Of classic optimization problems, the traveling salesman problem has received the most intensive study. To test the power of simulated annealing, we used the algorithm on traveling salesman problems with as many as several thousand cities. This work is described in a final section, followed by our conclusions.

Combinatorial Optimization

The subject of combinatorial optimization (1) consists of a set of problems that are central to the disciplines of computer science and engineering. Research in this area has a distinctive flavor: these problems are discrete, often combinatorial, and often involve the finding of a minimum or maximum value of a function of very many (often infinite) variables. This function is usually called the cost function or objective function; it represents a quantitative measure of the "goodness" of some configuration. The cost function depends on the detailed configuration of the many parts of the system. We are more familiar with optimization problems occurring in the physical design of computers, but examples used below are drawn from other contexts.

Summary. There is a deep and useful connection between statistical mechanics and optimization. The central constructs in combinatorial optimization (finding the minimum of a given function depending on many parameters) is depicted in a way that provides a framework for optimization of very large and complex systems. This connection to statistical mechanics expresses new information and provides an alternative perspective on traditional optimization problems and methods.

that context. The number of variables involved may range up into the tens of thousands.

The classic example, because it is so simply stated, is of combinatorial optimization. Given a list of N cities and a matrix of distances between every pair of cities, one must find a tour that visits every city once and returns finally to the starting point, minimizing the total distance. Problems with this flavor arise in all areas of scheduling and design. Two subfamily problems are of general interest: predicting the expected cost of the minimum expected cost, and computing the minimum cost of a given configuration.

Some class of typical arrangements of cities, and sometimes an ordering of the cities, is given. It is necessary to determine the minimum cost of the arrangement given necessary to determine the minimum cost. All exact methods known for determining an optimal route require a computer that can store the entire route.

with N , so that in practice exact solutions can be attempted only on problems involving a few hundred cities or less.

The traveling salesman problem is the large class of NP-complete problems, which has received extensive study in the past 30 years (2). No method for exact solutions with a computing time bounded by a power of N has been found for all of these problems, but if such a solution were found, it could be adapted into a procedure for solving all members of the class. It is not known what features of the individual problems in the NP-complete class are the cause of their difficulty.

Since the NP-complete class of problems contains many situations of practical interest, heuristic methods have been developed with computational requirements proportional to some power of N . Heuristics are neither problem-specific; there is no guarantee that a heuristic procedure for finding near-optimal solutions for one NP-complete problem will be effective for another.

There are two basic strategies for heuristic optimization: "divide-and-conquer" and "stochastic improvement." In the first, one divides the problem into subproblems of manageable size, then solves the subproblems. The solution to the subproblems are then combined to produce the solution to the original problem. For the method to produce very good solutions, the subproblems must be naturally disjoint, and the division made must be an appropriate one, so that errors made in patching do not offset the gains.

S. Kirkpatrick and C. D. Gelatt, Jr. are research associates of AT&T Bell Laboratories, Murray Hill, New Jersey. M. P. Vecchi is a research associate of AT&T Bell Laboratories, Holmdel, New Jersey. This work was supported by the Office of Naval Research, Arlington, Virginia, under grant N00014-82-K-0001.

Lisi Qarkaxhija

Algorithms, AI & Data Science II

Lecture 16: Markov Chain Monte Carlo Algorithms

July 2, 2025

20

Notes: